

biomaRt

Kasper D. Hansen

- [Dependencies](#)
- [Corrections](#)
- [Overview](#)
- [Other Resources](#)
- [Specifying a mart and a dataset](#)
- [Building a query](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(biomaRt)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("biomaRt"))
```

Corrections

Improvements and corrections to this document can be submitted on its [GitHub](#) in its [repository](#).

Overview

We use a large number of different databases in computational biology. “Biomart” is a flexible interface to a biological database. The idea is that any kind of resource can setup a Biomart and then users can access the data using a single set of tools to access multiple databases.

The *[biomaRt](#)* package implements such an interface.

Databases supporting the BiomaRt interface includes Ensembl (from EBI), HapMap and UniProt.

Other Resources

- The vignette from the [biomaRt webpage](#).

Specifying a mart and a dataset

To use *biomaRt* you need a mart (database) and a dataset inside the database. This is somewhat similar to *AnnotationHub*.

```
head(listMarts())
```

```
##           biomart           version
## 1      ensembl  ENSEMBL GENES 81 (SANGER UK)
## 2          snp   ENSEMBL VARIATION 81 (SANGER UK)
## 3    regulation ENSEMBL REGULATION 81 (SANGER UK)
## 4          vega          VEGA 61 (SANGER UK)
## 5    fungi_mart_28    ENSEMBL FUNGI 28 (EBI UK)
## 6 fungi_variations_28 ENSEMBL FUNGI VARIATION 28 (EBI UK)
```

```
mart <- useMart("ensembl")
mart
```

```
## Object of class 'Mart':
## Using the ensembl BioMart database
## Using the dataset
```

```
head(listDatasets(mart))
```

```
##                                dataset
## 1      oanatinus_gene_ensembl
## 2      cporcellus_gene_ensembl
## 3      gaculeatus_gene_ensembl
## 4      lafricana_gene_ensembl
## 5      itridecemlineatus_gene_ensembl
## 6      choffmanni_gene_ensembl
##                                description version
## 1      Ornithorhynchus anatinus genes (OANA5)  OANA5
## 2      Cavia porcellus genes (cavPor3)  cavPor3
## 3      Gasterosteus aculeatus genes (BROADS1)  BROADS1
## 4      Loxodonta africana genes (loxAfr3)  loxAfr3
## 5      Ictidomys tridecemlineatus genes (spetri2)  spetri2
## 6      Choloepus hoffmanni genes (choHof1)  choHof1
```

```
ensembl <- useDataset("hsapiens_gene_ensembl", mart)
ensembl
```

```
## Object of class 'Mart':
## Using the ensembl BioMart database
## Using the hsapiens_gene_ensembl dataset
```

You can see that the different datasets are organized by species; we have select *Homo sapiens*.

You access this database over the internet. Sometimes you need to specify a proxy server for this to work; details are in the [biomaRt](#) vignette; I have never encountered this.

Building a query

There is one main function in [biomaRt](#): `getBM()` (get Biomart). This function retrieves data from a Biomart based on a query. So it is important to understand how to build queries.

A Biomart query consists of 3 things: “attributes”, “filters” and “values”. Let us do an example. Let us say we want to annotate an Affymetrix gene expression microarray. We have Affymetrix probe ids in R and we want to retrieve gene names. In this case gene names is an “attribute” and Affymetrix probe ids is a “Filter”. Finally, the “values” are the actual values of the “filter”, ie. the ids.

An example might be (not run)

```
values <- c("202763_at", "209310_s_at", "207500_at")
getBM(attributes = c("ensembl_gene_id", "affy_hg_u133_plus_2"),
      filters = "affy_hg_u133_plus_2", values = values, mart = ensembl)
```

```
##      ensembl_gene_id affy_hg_u133_plus_2
## 1 ENSG00000196954      209310_s_at
## 2 ENSG00000137757      207500_at
## 3 ENSG00000164305      202763_at
```

Note that I list `affy_hg_u133_plus_2` under both `attributes` and `filters`. It is listed under `attributes` because otherwise it would not appear in the return value of the function. If I don't have the `affy_hg_u133_plus_2` column in my `data.frame` I wouldn't know where genes are mapped to which probe ids. I would just have a list of which genes were measured on the array.

An example of a filter that might not appear in the `attributes` is if you want to only select autosomal genes. You may not care about which chromosomes the different genes appear on, just that they are on autosomal chromosomes.

Important note: Biomart (at least Ensembl) logs how often you query. If you query too many times, it disables access for a while. So the trick is to make a single vectorized query using a long list of values and not call `getBM()` for each individual value (doing this is also much, much slower).

A major part of using *biomaRt* is figuring out which attributes and which filters to use. You can get a description of this using `listAttributes()` and `listFilters()`; both return a very long `data.frame`.

```
attributes <- listAttributes(ensembl)
head(attributes)
```

```
##           name           description
## 1 ensembl_gene_id Ensembl Gene ID
## 2 ensembl_transcript_id Ensembl Transcript ID
## 3 ensembl_peptide_id Ensembl Protein ID
## 4 ensembl_exon_id Ensembl Exon ID
## 5 description Description
## 6 chromosome_name Chromosome Name
```

```
nrow(attributes)
```

```
## [1] 1210
```

```
filters <- listFilters(ensembl)  
head(filters)
```

```
##           name      description  
## 1 chromosome_name Chromosome name  
## 2           start Gene Start (bp)  
## 3           end   Gene End (bp)  
## 4    band_start      Band Start  
## 5    band_end      Band End  
## 6 marker_start      Marker Start
```

```
nrow(filters)
```

```
## [1] 306
```

A lot of the attributes are gene names for the corresponding gene in a different organism. All these entries makes it a bit hard to get a good idea of what is there.

In Biomart, data is organized into pages (if you know about databases, this is a non-standard design). Each page contains a subset of attributes. You can get a more understandable set of attributes by using pages.

```
attributePages(ensembl)
```

```
## [1] "feature_page" "structure"      "homologs"      "snp"  
## [5] "snp_somatic"  "sequences"
```

```
attributes <- listAttributes(ensembl, page = "feature_page")  
head(attributes)
```

##	name	description
## 1	ensembl_gene_id	Ensembl Gene ID
## 2	ensembl_transcript_id	Ensembl Transcript ID
## 3	ensembl_peptide_id	Ensembl Protein ID
## 4	ensembl_exon_id	Ensembl Exon ID
## 5	description	Description
## 6	chromosome_name	Chromosome Name

```
nrow(attributes)
```

```
## [1] 184
```

All the homologs I complain about above are part of the ... homologs page.

An attribute can be part of multiple pages. It turns out that `getBM()` can only return a query which uses attributes from a single page. If you want to combine attributes from multiple pages you need to do multiple queries and then merge them.

Another aspect of working with `getBM()` is that sometimes the return `data.frame` contains duplicated rows. This is a consequence of the internal structure of the database and how queries are interpreted.

The *biomaRt* vignette is very useful and readable and contains a lot of example tasks, which can inspire future use. As a help, I have listed some of them here:

1. Annotate a set of Affymetrix identifiers with HUGO symbol and chromosomal locations of corresponding genes.
2. Annotate a set of EntrezGene identifiers with GO annotation.
3. Retrieve all HUGO gene symbols of genes that are located on chromosomes 17, 20 or Y, and are associated with one the following GO terms: "[GO:0051330](#)", "[GO:0000080](#)", "[GO:0000114](#)", "[GO:0000082](#)".
4. Annotate set of identifiers with INTERPRO protein domain identifiers.
5. Select all Affymetrix identifiers on the hgu133plus2 chip and Ensembl gene identifiers for genes located on chromosome 16 between basepair 1100000 and 1250000.
6. Retrieve all entrezgene identifiers and HUGO gene symbols of genes which have a "MAP kinase activity" GO term associated with it.
7. Given a set of EntrezGene identifiers, retrieve 100bp upstream promoter sequences.
8. Retrieve all 5' UTR sequences of all genes that are located on chromosome 3 between the positions 185514033 and 185535839
9. Retrieve protein sequences for a given list of EntrezGene identifiers.
10. Retrieve known SNPs located on the human chromosome 8 between positions 148350 and 148612.

11. Given the human gene TP53, retrieve the human chromosomal location of this gene and also retrieve the chromosomal location and RefSeq id of it's homolog in mouse.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] methods      stats      graphics  grDevices  utils      datasets  base
##
## other attached packages:
## [1] biomaRt_2.24.0 BiocStyle_1.6.0 rmarkdown_0.7
##
## loaded via a namespace (and not attached):
## [1] IRanges_2.2.7      XML_3.98-1.3      digest_0.6.8
## [4] bitops_1.0-6      GenomeInfoDb_1.4.2 DBI_0.3.1
## [7] stats4_3.2.1      formatR_1.2       magrittr_1.5
## [10] RSQLite_1.0.0     evaluate_0.7.2    stringi_0.5-5
## [13] S4Vectors_0.6.3   tools_3.2.1       stringr_1.0.0
## [16] Biobase_2.28.0    RCurl_1.95-4.7    parallel_3.2.1
## [19] yaml_2.1.13       BiocGenerics_0.14.0 AnnotationDbi_1.30.1
## [22] htmltools_0.2.6   knitr_1.11
```

